



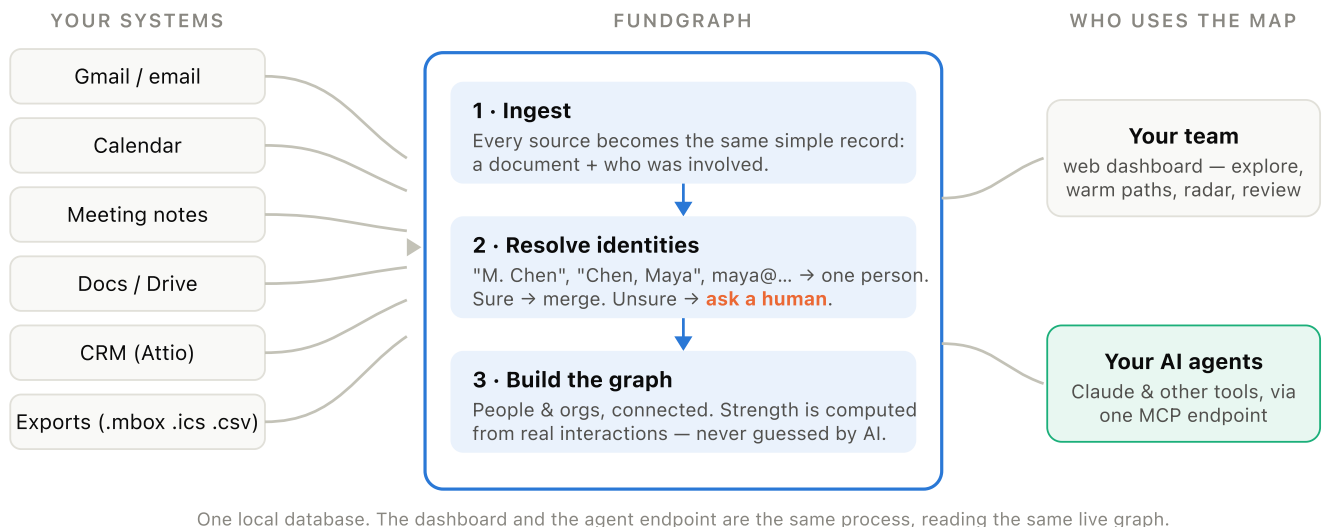
How it works — one document for your whole team, technical or not.

THE PROBLEM

A fund's knowledge is scattered across **email, calendars, meeting notes, docs, and the CRM**. The same person appears a hundred different ways. Nobody can see the whole picture — and neither can AI: an assistant that guesses with keyword or "similarity" search fetches the wrong things and doesn't know what exists.

WHAT FUNDGRAPH DOES

fundgraph reads all of those sources and builds one **map of reality**: every person and organization, recognized across sources as one entity, connected by relationship strength **computed from actual behavior** — how often you really meet, email, and work together. Your team browses the map in a dashboard; your AI tools plug into the exact same map and stop guessing.



In one sentence: fundgraph turns your fund's scattered communication into a single, trustworthy map of *who you know and how well* — that both people and AI can navigate.

WHAT'S INSIDE

- Page 2** The pipeline, step by step — how scattered data becomes one graph, with a human in the loop.
- Page 3** What you get — warm paths, radar, briefs & fund memory, agents. Plus: real questions it answers.
- Page 4** The ideas, in plain language — what a knowledge graph is, and why "AI document search" (RAG) isn't enough.
- Page 5** How agents connect (MCP, explained simply) — and the technology stack, including what Postgres is.
- Page 6** Trust — privacy layers, where AI is allowed and where it's banned, and the technical fact sheet.

The pipeline, step by step

1 Ingest — everything becomes one simple shape

Drop in a Gmail export, a calendar file, a CSV of contacts, or connect Attio live. Every source is reduced to the same record: *a document (an email, a meeting, a note) plus the people and organizations involved.*

Technical: adapters for .mbox / .ics / .csv / .jsonl, Granola, Attio, Gmail/Calendar/Drive APIs. Ingest is idempotent — re-importing the same data updates in place, never duplicates, and never destroys review history.

2 Entity resolution — is "M. Chen" the same person as "Maya Chen"?

Every name and address is matched against known people using several signals (same email, same name at the same firm, and so on). The score decides what happens next — and **a tie always goes to a human**:



Real example — one person, five appearances: Maya Chen <maya@nordwind.vc> · "Chen, Maya" (reversed, in an email header) · an encoded form from a calendar invite · an Attio CRM contact — merged automatically. "M. Chen" <mchen@gmail.com> (a personal address, score 0.83) — **asked a human first**, then merged. One node on the map.

Technical: blocking → candidate generation → probabilistic scoring → human review queue. Conflicting evidence (same name, different work domain) always asks. Decisions are recorded, audited, and replayed if the graph is ever rebuilt from scratch — a click is never lost.

3 Relationship strength — computed, with receipts

Every connection's strength comes from a transparent recipe over real events — recent contact counts for more, and every score can show the evidence behind it ("3 meetings, 2 emails, 1 co-authored doc");

3 meetings x3 + 2 direct emails x2.5 + 1 shared doc x1.5 →
recency decay (180-day half-life) → **strength 74%**

Technical: $\text{weight}(\text{kind}) \times \text{decay}(\text{age})$, saturating via $1 - e^{-w/6}$ so evidence accumulates but never exceeds 100%. All weights are editable in Settings per firm; saving rebuilds the graph instantly. The graph is a read model — deterministically rebuilt, never hand-edited.

4 Serve — one map, two doors

The finished graph is served two ways from the same process: a **dashboard** for people and an **MCP endpoint** for AI agents. Both see the same live data; a review decision made in one is instantly true in the other.

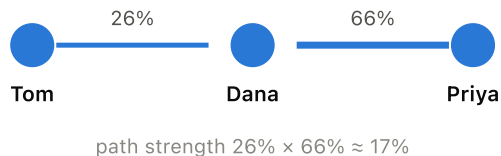
Technical: single Node process; embedded Postgres (zero setup) or your own `DATABASE_URL`. MCP is served over Streamable HTTP at `/mcp`.

The trust rule: the system never merges two identities on a guess, and no AI model ever assigns a relationship score. Anything uncertain is a question for a human — that's what makes the map safe to act on.

What you get out of it

Warm paths — the best route to an introduction

Ask "how do I get to Priya?" and the graph finds the strongest chain of real relationships, plus a ranked list of who should make the intro — scored by their *weaker* leg, because an introducer is only as good as their weaker relationship.



Radar — who needs attention *now*

Every relationship has its own rhythm. Radar learns each pair's usual contact interval, so "overdue" is relative: three quiet weeks are nothing for a quarterly contact and alarming for a weekly one.

overdue · Priya ↔ Maya — last contact 16d ago · usually every 10d · **6d overdue**

Briefs & fund memory

Click any person: strongest relationships with the evidence behind each score, recent shared documents, and the firm's deal history with their company — **including the passes and the stated reasons**, so "have we seen this company before, and why did we say no?" has an answer.

Agents on the graph — via MCP

MCP is the open standard that lets AI assistants call tools. fundgraph exposes the whole map as one endpoint, so Claude (or any MCP client) answers from **facts in your graph** instead of guessing.

You: "Prep me for my meeting with Priya Nair. I'm Dana."

Claude: calls `meeting_prep` → gets her profile, your relationship history with receipts, shared documents, your warm paths and best introducers → writes the brief.

Structured facts from the graph; prose from the model; nothing hallucinated.

<code>meeting_prep</code> —	<code>company_memory</code> —
everything for one meeting	deals, incl. passes
<code>find_warm_path</code> ·	<code>relationship_radar</code> —
<code>find_introducers</code>	timing
<code>entity_brief</code> ·	<code>review_queue</code> ·
<code>search_entities</code>	<code>review_resolve</code>

Connect once, while the dashboard is running:

```
claude mcp add --transport http fundgraph  
http://localhost:4321/mcp
```

Human controls, in the same UI

The review queue (confirm/reject uncertain matches), manual merge/unmerge, per-firm scoring weights, automated-sender detection (newsletters and robots stay out of the graph), and a full audit trail of every change.

Why not just vector search? Similarity search retrieves *text that sounds related*; relationship questions need a graph with verified identities. Full plain-language comparison on page 4.

Ask it like a person — examples

"Who should intro us to Priya Nair at Meridian?"

Warm path tab → ranked introducers · agent tool `find_introducers`

"Which relationships are quietly going cold this month?"

Radar tab · agent tool `relationship_radar`

"Have we seen Copperline Metrics before — and why did we pass?"

Company brief → deal history · agent tool `company_memory`

"Prep me for tomorrow's meeting with Priya. I'm Dana."

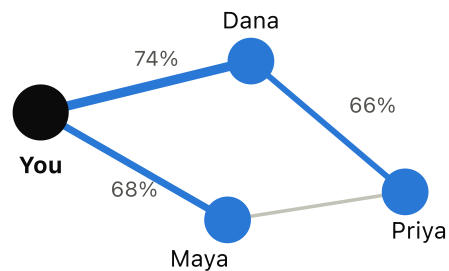
Person brief in the dashboard · agent tool `meeting_prep`

The ideas, in plain language

What is a "knowledge graph"?

A graph is just **things and the connections between them** — circles and lines. Here the circles are people and organizations; every line is a relationship, carrying a strength score and the evidence behind it. Think of it as **a map assembled from your documents**: the documents stay where they are, and the map records what they prove — who exists, who knows whom, how well, since when.

Under the hood there's no exotic technology: the map is ordinary database tables — one for people & orgs, one for connections, one for the documents behind them. "Graph" describes the shape of the data, not a special machine.



line weight = how strong, with receipts

Three ways to point AI at your fund's data

1 · The model alone

Claude/ChatGPT know nothing about *your* fund. Asked anyway, they answer fluently — from nothing. Confident guessing.

2 · "AI document search" (RAG)

The model searches your files for passages that *sound similar* to the question, reads the top few, answers. Great for "find the clause". Weak when the answer requires knowing who's who, weighing *all* the evidence, or noticing what's missing.

3 · A verified map (fundgraph)

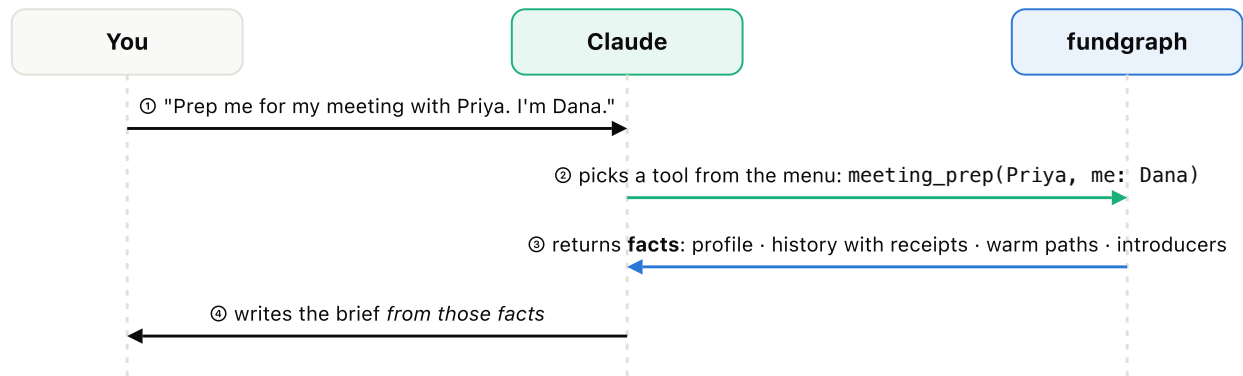
The facts are extracted, identity-checked, and connected *before* any question is asked. The model then queries the finished map and writes prose from real numbers.

The question	Searching a pile of text (RAG)	Asking the map (fundgraph)
"Who can intro me to Priya?"	Returns emails that <i>mention</i> Priya. Whether the senders actually know her — and how well — is a guess.	Computes the strongest chain of real relationships (Tom → Dana 26% → Priya 66%) and ranks introducers by their weaker leg.
"How strong is our Meridian relationship?"	Whatever the top few retrieved paragraphs imply — one lucky email can dominate the answer.	Sums <i>every</i> touchpoint (meetings, emails, docs), weighted and time-decayed — with the receipts listed.
"Is M. Chen the same person as Maya Chen?"	To text search these are just different strings. It cannot know, so it silently treats them as two people.	Identity resolution scored the match (0.83), asked a human, and merged. The map has one Maya.
"Have we seen this company before?"	Maybe surfaces an old mention — if you phrase the query luckily. Silence proves nothing.	Deal history is recorded structurally: every INVEST and every PASS with its stated reason and source document.

Not either/or. Document search is still the right tool for "find the clause in the LPA". fundgraph exists because **relationship questions need structure** — and once the map exists, the AI's answers to those questions stop being guesses. The two compose: facts from the graph, prose from the model.

How agents connect — MCP, in plain language

MCP (Model Context Protocol) is an open standard — think **USB-C for AI**. Instead of a custom integration per app, any assistant that speaks MCP (Claude Desktop, Claude Code, Cursor, and others) can plug into any MCP "server". fundgraph is such a server: it offers the assistant a menu of eleven named **tools** — small, well-defined actions like `meeting_prep` or `find_warm_path` — each with documented inputs and outputs. The assistant reads the menu and decides, per question, which tool answers it.



The assistant never rummages the database — it can only call the listed tools, and every answer passes the same privacy filters.

Why this matters: the model contributes language and judgment; the numbers, names, and paths come from the graph. If the graph doesn't know something, the honest answer is "no path found" — not an invented one.

Setup is one line while the dashboard runs:

```
claude mcp add --transport http fundgraph http://localhost:4321/mcp
```

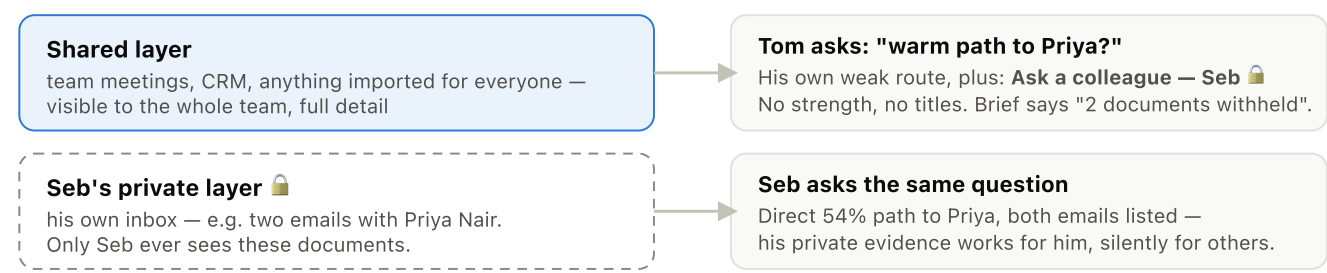
Add `?as=Seb%20Hopkins` to give an agent one member's private view — it then sees exactly what that person would see, nothing more.

What it's built on

The application	JavaScript on Node.js — the same mainstream technology behind most modern web services. One program, ~no external services, runs on a laptop. Open source (MIT), so anyone can audit exactly what it does with your data.
The database — and what "Postgres" means	PostgreSQL ("Postgres") is one of the world's most trusted database systems — ~30 years old, free, and used by banks, airlines, and most tech companies. A database is simply a program that stores structured records and answers questions about them reliably (think: industrial-strength spreadsheets with a query language). fundgraph ships with an embedded Postgres (PGLite) — it runs <i>inside</i> the app, keeps everything in one folder on your machine, and needs zero installation. When a team outgrows one laptop, point <code>DATABASE_URL</code> at a normal Postgres server — same data, same code.
The dashboard	Plain HTML/CSS/JavaScript with D3 , the standard library for interactive data graphics — that's the force-directed network you see. No heavy framework; the whole front-end is three files.
AI — used narrowly	Anthropic's Claude , called only in the extraction step to read document text and propose candidates (people, orgs, deal signals) for the deterministic pipeline to verify. The graph, scoring, paths, and radar all work with the AI switched off.
Agent interface	The official MCP SDK — one endpoint, eleven tools, served by the same process as the dashboard (or standalone over stdio for headless setups).

Privacy layers — share the map, not the mail

The objection every partner raises: *"I'm not putting my inbox where the whole team can read it."*
fundgraph's answer: each member can keep sources in a **private layer**. The team learns that a connection *exists* and who to ask — never the evidence behind it.



The principle: **existence is shared, evidence is private**. Enforced server-side on every query (a standing "leak probe" test suite verifies it). Honest boundary: this is a cooperative model for one trusted team on one machine — not a defense against someone with direct access to the database file.

Where AI is used — and where it's banned

AI is used for reading, not deciding. An LLM mines document *text* ("their CFO Ruth Calloway flagged...") and proposes people, orgs, and deal signals no structured field ever sees. Every proposal is then checked by plain code: the name must literally appear in the text, emails must match exactly, low confidence is dropped — and the survivors go through the **same** resolution-and-review pipeline as everything else.

AI is banned from scoring. Models confidently misjudge relationships, so no LLM ever sets a strength, merges an identity, or invents a contact. The LLM proposes; deterministic code and humans dispose. That division is what makes the graph safe for agents to act on.

Quick reference

Run it	<code>npm install && npm start</code> → <code>http://localhost:4321</code> . First run offers a one-click fictional sample world — the graph, a pending review, a team with a private layer — so you can try everything before connecting real data.
Data in	Gmail Takeout (.mbox, streamed — multi-GB is fine), .ics, .csv, .jsonl, Granola cache, Attio live sync, Gmail/Calendar/Drive APIs. All ingestion is idempotent — re-importing never duplicates.
Agents	MCP over Streamable HTTP at <code>/mcp</code> (11 tools), or stdio (<code>fundgraph mcp</code>). Per-member scoping: <code>?as=<member></code> binds an agent to that person's private layer.
Trust & audit	Deterministic scoring with receipts · human review for uncertain merges · decisions replayed on rebuilds · full audit trail · automated-sender filtering (robots stay out of the graph) · privacy leak-probe in the test suite.
Stack	Node.js 20+ · embedded Postgres (PGLite), or <code>DATABASE_URL</code> for a shared server · D3 dashboard · Anthropic Claude (extraction only) · MCP SDK — plain-language tour on page 5. MIT open source.